NORTH BENGAL INTERNATIONAL UNIVERSITY

Lab Report : 7

TOPIC : FIND A REAL ROOT OF THE EQUATION X^3-2X-5 USING NEWTON RAPHSON METHOD WHERE A=2, B=3 AND TOLERANCE IS INPUT BY USER

COURSE CODE : MATH 0541-2212

COURSE TITLE : NUMERICAL ANALYSIS SESSIONAL

Submitted By:

MD. TANVIR RAHMAN PRANTO

0712340012101005

CSE -26

Submitted To:

SAIFUR RAHMAN

LECTURER, CSE

Find a real root of the equation x^3-2x-5 using Newton Raphson method where a=2, b=3 and tolerance is input by user:

Theory of the Newton-Raphson Method:

The Newton-Raphson method is a powerful and efficient numerical technique for finding the roots of a real-valued function. It relies on the iterative application of a formula derived from the first-order Taylor series expansion of the function.

Concept:

- The method starts with an initial guess x_0 for the root.
- It uses the derivative of the function to refine this guess iteratively.

Formula: The root approximation formula is given by:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

where:

- x_n is the current approximation.
- $f(x_n)$ is the value of the function at x_n.
- $f'(x_n)$ is the value of the derivative of the function at x_n.

Steps:

1. **Initial Guess**: Start with an initial guess x_0.
2. **Iterative Refinement**: Compute the next approximation x_{n+1} using the formula.
3. **Convergence**: Repeat the process until the difference between successive approximations is within a specified tolerance.

Algorithm:

Algorithm for Finding a Real Root of the Equation x^3 - 2x - 5 Using Newton-Raphson Method

1. Start
2. Define the function representing the equation:
- f(x) = x^3 - 2x - 5
3. Define the derivative of the function:
- f_prime(x) = 3x^2 - 2
4. Input the initial guess (x0) and the tolerance:
- Let x0 be 2.5 (between a = 2 and b = 3)
- Prompt the user to input the tolerance

5. Initialize the iteration counter:
- Set step = 0

6. Iterative process:
- While True:
  - Calculate the new approximation: x1 = x0 - f(x0) / f_prime(x0)
  - Increment the iteration counter: step = step + 1
  - Print the current step and the new approximation: x1
  - If the absolute difference between the new and previous approximation (|x1 - x0|) is less than the tolerance, then:
    - Break the loop
  - Update the previous approximation: x0 = x1

7. Output the final result:
- Print the root: x1

8. End

Code:

```
import math
def f(x):
    return x**3 - 2*x - 5

# Derivative of the function
def f_prime(x):
    return 3*x**2 - 2

# Newton-Raphson method
def newton_raphson(x0, tol):
    step = 0
    while True:
        x1 = x0 - f(x0) / f_prime(x0)
        step += 1
        print(f"Step {step}: x = {x1:.6f}")
        if abs(x1 - x0) < tol:
            break
        x0 = x1
    return x1

# Initial guess
x0 = 2.5  # Starting point between a=2 and b=3

# Input tolerance
tolerance = float(input("Enter the tolerance: "))

# Finding the root
root = newton_raphson(x0, tolerance)

print(f"The root is approximately: {root:.6f}")
```

Output:

```
OUTPUT    TERMINAL

PS C:\Users\Lenovo> & C:/Users/Lenovo/AppData/Local/Programs/Python/Python313/
Enter the tolerance: 0.000002
Step 1: x = 2.164179
Step 2: x = 2.097135
Step 3: x = 2.094555
Step 4: x = 2.094551
Step 5: x = 2.094551
The root is approximately: 2.094551
PS C:\Users\Lenovo>
```

Conclusion: This Python code efficiently finds the real root of the equation x^3- 2x -5 using the Newton-Raphson method. It iteratively refines the root estimate, ensuring rapid convergence. The structure ensures clarity and ease of use, making it an effective tool for root-finding.